

Análisis de Algoritmos

Práctica: STL de C++

Instrucciones Responder según tu intuición, posteriormente probar y finalmente consultar `cppreference.com` para verificar.

1. En clase vimos que `sort` puede recibir como tercer parámetro una función de comparación. Imagina que tienes el siguiente vector de pares:

```
vector<pair<int,string>> estudiantesUCB;
```

que almacena el año de nacimiento y el nombre de estudiantes de segundo semestre de la UCB. Responde a las siguientes preguntas:

- Si quiero ordenar a los estudiantes de más joven a menos joven, ¿cómo debería llamar a la función `sort`?
- Si quiero ordenar alfabéticamente por nombre, ¿cómo debería llamar a `sort`?
- ¿Y si quisiera ordenar por nombre y, en caso de empate, por año?
- ¿Cómo debería llamar a `find` si quisiera encontrar un estudiante del vector dado un nombre, sin saber su año de nacimiento? ¿Es posible pasarle un comparador propio?
- Imagina que tienes el siguiente vector:

2000 Ana	2002 Tatiana	2002 Zara	2003 Alicia	2004 Bernardo	2004 Santiago	2005 Javier
[0]	[1]	[2]	[3]	[4]	[5]	[6]

¿Cómo podrías usar `upper_bound` o `lower_bound` para encontrar la posición de un elemento dado un nombre?

Si ejecutas `sort(vec.begin()+1, vec.end()-2, comparadorPorAo)`, donde `comparadorPorAño` devuelve `true` si el año de nacimiento del primer par es que el segundo, ¿cómo quedaría el vector?

2. `find` retorna el iterador a la **primera** ocurrencia de un valor específico. ¿Cómo podrías encontrar el número de elementos que cumplen una determinada condición, por ejemplo, la cantidad de estudiantes que nacieron después del 2002? Entrar a https://cplusplus.com/reference/algorithm/find_if/ y desarrollar la función correspondiente.
3. Dado un vector de enteros, compara las siguientes dos estrategias:

```
// Opcion A
sort(vec.begin(), vec.end());
reverse(vec.begin(), vec.end());

// Opcion B
sort(vec.begin(), vec.end(), greater<int>());
```

¿El resultado es siempre idéntico? ¿Cuál es la complejidad temporal de cada opción? ¿Hay algún caso donde el resultado *no* sea el mismo?

4. Averigua qué hace la función `stable_sort` de C++. ¿Cuál es su complejidad temporal y en qué se diferencia del `sort` normal? Da un ejemplo concreto donde importaría usar uno u otro.
5. ¿`queue` y `stack` deberían tener iteradores? ¿Por qué?
6. `binary_search` retorna un `bool`. Usando `lower_bound` y `upper_bound`, ¿cómo contarías cuántas veces aparece un número en un vector ordenado **sin** recorrerlo linealmente? ¿Existe alguna función de la STL que combine ambas en una sola llamada?

Ejercicios

1. Utilizando solamente funciones de la STL, dado un vector de cadenas con nombres de países, encuentra cuántos países empiezan con la letra 'A'.
2. Tenés una lista de transacciones bancarias con monto y descripción:

```
vector<pair<int,string>> transacciones = {  
    {200,"Supermercado"}, {-50,"Retiro"},  
    {200,"Deposito"},     {-200,"Pago servicio"},  
    {150,"Transferencia"},{-50,"Comision"}  
};
```

- Encuentra la primera transacción negativa.
 - Utiliza `stable_sort` para ordenar por monto. ¿Qué orden relativo tienen entre sí los dos retiros de -50? ¿Y si usaras `sort` normal?
 - Encuentra el número de transacciones mayores a 100.
3. Dado el siguiente vector

```
vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
```

, sin usar un ciclo explícito, solamente funciones de la STL, elimina todos los elementos duplicados dejando solo la primera aparición de cada valor, manteniendo el orden relativo.